

Java Collections: List Family - Cheat Sheet

The List family includes `ArrayList`, `LinkedList`, `Vector`, `Stack`, and `CopyOnWriteArrayList`.

The collection hierarchy (brief)

Everything in `java.util` traces back to `Iterable`. Then `Collection` splits into 3 main branches.

```
Iterable
|
+-- Collection
|   |
|   +-- List      (ordered, indexed, duplicates ok)
|   |   |
|   |   +-- ArrayList
|   |   +-- LinkedList (also implements Deque)
|   |   +-- Vector     (legacy, synchronized)
|   |   +-- Stack      (legacy, LIFO)
|   |   +-- CopyOnWriteArrayList
|   |
|   +-- Set        (no duplicates)
|   +-- Queue      (FIFO/priority semantics)
|   +-- Deque      (double-ended)
```

`List` is the branch you reach for when order matters and duplicates are fine. If you don't have a specific reason to pick something else, `ArrayList` is the default.

The List interface

`List` is the contract. It guarantees:

- Elements have a defined order (insertion order by default).
- Elements are accessible by integer index.
- Duplicates are allowed.
- Nulls are allowed (most implementations, except `List.of`).

Key methods you'll use every day:

```

List<String> names = new ArrayList<>();
names.add("alice");           // append
names.add(0, "bob");          // insert at index
String first = names.get(0);  // read by index
names.set(1, "carol");        // replace at index
names.remove(0);              // remove by index (returns removed)
names.remove("carol");        // remove by value (returns boolean)
int idx = names.indexOf("alice"); // first occurrence, -1 if absent
int last = names.lastIndexOf("alice");
boolean has = names.contains("alice");
int n = names.size();
boolean empty = names.isEmpty();
names.clear();
List<String> view = names.subList(0, 2); // view, not a copy

```

A quick word on the 3 ways to create a list:

```

List<String> a = new ArrayList<>();           // mutable, empty
List<String> b = Arrays.asList("x", "y");     // fixed-size, backed by array,
// allows set() but not add()/remove()
List<String> c = List.of("x", "y");           // truly immutable (Java 9+), throws
// UnsupportedOperationException on any mutation

```

People mix these up constantly. `Arrays.asList` is the classic trap. You can call `set()` on it but `add()` blows up at runtime.

ArrayList

The workhorse. It's a resizable array under the hood.

Defaults and growth:

- Initial capacity is 10 (when you call `new ArrayList<>()` and then add the first element).
- When full, the backing array grows by roughly 1.5x (Oracle JDK uses `oldCapacity + (oldCapacity >> 1)`).
- If you know the size up front, preallocate: `new ArrayList<>(expectedSize)`. This skips multiple grow-and-copy cycles.

Complexity:

- `get(i)` : $O(1)$, direct array index.
- `add(e)` at the end: $O(1)$ amortized. Occasionally pays for a resize.
- `add(i, e)` in the middle: $O(n)$, has to shift everything to the right.
- `remove(i)` : $O(n)$, shifts everything left.
- `contains(o)` / `indexOf(o)` : $O(n)$, linear scan.

Iteration is fast because the array sits in contiguous memory. CPU cache loves it.

```
List<Order> orders = new ArrayList<>(1000);    // preallocated
for (int i = 0; i < 1000; i++) {
    orders.add(new Order(i));
}
Order o = orders.get(500);                    // O(1)
orders.add(new Order(1001));                  // O(1) amortized
orders.add(0, new Order(-1));                 // O(n), shifts everything
orders.remove(500);                           // O(n)
```

LinkedList

A doubly-linked list. Each element is a node with `prev` and `next` pointers.

It implements both `List` and `Deque`, which is the one place it shines.

Complexity:

- `get(i)` : $O(n)$. The list has to walk from the head (or tail, whichever is closer) to index i .
- add at head or tail: $O(1)$.
- `add(i, e)` : $O(n)$ because of the walk to find position i . The actual insertion is $O(1)$ once you're there.
- `remove(i)` : $O(n)$ for the same reason.

Memory overhead per element is high. Each node is a separate object with 2 pointers plus the value.

Why it's almost always the wrong choice:

- Cache locality kills it. Walking a linked list means chasing pointers across the heap.
- Even the "fast middle insert" case loses to `ArrayList` because the seek dominates the work.
- Benchmarks consistently show `ArrayList` beating `LinkedList` on real workloads, even when the workload looks like a `LinkedList` favorite.

When `LinkedList` actually earns its place: you need both head and tail operations (queue or deque behavior) and you don't want `ArrayDeque` for some reason. `ArrayDeque` is usually faster, so this case is narrow.

```
LinkedList<Task> tasks = new LinkedList<>();
tasks.addFirst(new Task("urgent"));    // O(1)
tasks.addLast(new Task("later"));      // O(1)
Task next = tasks.pollFirst();         // O(1)

// Don't do this on a large LinkedList:
Task middle = tasks.get(5000);         // O(n), walks 5000 nodes
```

Vector

Legacy. `Vector` predates the collections framework. It shipped with Java 1.0.

Every method is `synchronized` . That sounds safe, but in practice:

- Contention under any real load. Every read and write grabs the same lock.
- Iteration isn't atomic anyway, so the synchronization gives a false sense of safety.

Modern replacements:

- `ArrayList` plus explicit synchronization where you need it.
- `Collections.synchronizedList(new ArrayList<>())` wraps an `ArrayList` with the same coarse locking, but at least it's explicit.
- `CopyOnWriteArrayList` for read-heavy concurrent workloads.

Don't use `Vector` in new code. The only time you'll touch it is when you inherit code that already uses it.

Stack

Also legacy. `Stack` extends `Vector` , so it carries all the same synchronization baggage.

LIFO API: `push` , `pop` , `peek` .

Use `ArrayDeque` instead. It's faster, unsynchronized, and has a cleaner API.

```
// Don't:
Stack<Integer> s = new Stack<>();

// Do:
Deque<Integer> stack = new ArrayDeque<>();
stack.push(1);
stack.push(2);
int top = stack.pop(); // 2
```

CopyOnWriteArrayList

Concurrent variant. Every write creates a brand new copy of the underlying array.

- Reads are lock-free and fast.
- Writes are expensive (full array copy).
- Iterators see a snapshot, so they never throw `ConcurrentModificationException` .

Best for read-mostly workloads like listener lists, config caches, event subscribers. Full treatment lives in `collections-concurrent.pdf` .

Time complexity table

Operation	ArrayList	LinkedList	Vector	CopyOnWriteArrayList
get(i)	O(1)	O(n)	O(1)	O(1)
add(end)	O(1) amortized	O(1)	O(1) amortized	O(n) (copies array)
add(index)	O(n)	O(n)	O(n)	O(n)
remove(index)	O(n)	O(n)	O(n)	O(n)
contains	O(n)	O(n)	O(n)	O(n)
iteration	O(n), fast	O(n), slow	O(n)	O(n), snapshot

`LinkedList`'s `add(end)` is O(1) because it keeps a tail pointer. Its `add(index)` is O(n) because the seek dominates. The `splice` itself is O(1).

ArrayList vs LinkedList in real code

The classic interview question. Most developers parrot “use `LinkedList` for middle inserts” without measuring anything.

The truth:

- Cache locality matters more than algorithmic complexity for small to medium sizes (which is almost every real workload).
- An `ArrayList` shift of 10,000 ints is cheaper than walking 10,000 `LinkedList` nodes, because the shift is one tight loop on contiguous memory.
- The crossover point where `LinkedList` wins is much further out than most people think, and usually doesn't exist in practice.

The decision rule:

1. Default to `ArrayList`.
2. If you need deque or queue behavior at both ends, look at `ArrayDeque` first, then `LinkedList`.
3. Only switch if you have a benchmark showing the change helps.

If someone tells you to use `LinkedList` without a benchmark, they're cargo-culting.

Iteration patterns

Several ways to walk a list. Pick based on what you need.

```

List<String> names = List.of("alice", "bob", "carol");

// 1. Classic indexed for loop. Use when you need the index.
for (int i = 0; i < names.size(); i++) {
    System.out.println(i + ": " + names.get(i));
}

// 2. Enhanced for (for-each). Compiles down to an Iterator.
for (String name : names) {
    System.out.println(name);
}

// 3. Explicit Iterator. Use when you need to remove during iteration.
Iterator<String> it = names.iterator();
while (it.hasNext()) {
    String name = it.next();
    if (name.startsWith("a")) {
        it.remove(); // safe
    }
}

// 4. ListIterator. Bidirectional, can also set() and add().
ListIterator<String> lit = names.listIterator();
while (lit.hasNext()) {
    String name = lit.next();
    if (name.equals("bob")) {
        lit.set("BOB");
    }
}

// 5. forEach with lambda.
names.forEach(name -> System.out.println(name));

// 6. Streams. Use when transforming/filtering.
List<String> upper = names.stream()
    .filter(n -> n.length() > 3)
    .map(String::toUpperCase)
    .collect(Collectors.toList());

```

Modifying during iteration

This is where `ConcurrentModificationException` bites people.

Standard list iterators are fail-fast. They check a `modCount` field on every `next()` call. If the list was modified through any path other than the iterator itself, you get a `CME`.

```

List<String> names = new ArrayList<>(List.of("a", "b", "c"));

// BAD. Throws ConcurrentModificationException.
for (String n : names) {
    if (n.equals("b")) {
        names.remove(n); // modifies list directly
    }
}

// GOOD. Use the iterator's remove().
Iterator<String> it = names.iterator();
while (it.hasNext()) {
    if (it.next().equals("b")) {
        it.remove();
    }
}

// BETTER. removeIf is concise and correct.
names.removeIf(n -> n.equals("b"));

// ALSO FINE. Collect targets first, then modify.
List<String> toRemove = names.stream()
    .filter(n -> n.equals("b"))
    .collect(Collectors.toList());
names.removeAll(toRemove);

```

Fail-fast vs fail-safe iterators get a deeper treatment in [collections-internals.pdf](#).

Sorting and searching

Three main entry points.

```

List<String> names = new ArrayList<>(List.of("charlie", "alice", "bob"));

// In-place sort, natural order.
Collections.sort(names); // pre-Java 8 style
names.sort(Comparator.naturalOrder()); // Java 8+ style

// Custom comparator.
names.sort(Comparator.reverseOrder());
names.sort((a, b) -> a.length() - b.length());

// Comparator chaining.
List<Order> orders = new ArrayList<>();
orders.sort(
    Comparator.comparing(Order::getCustomerId)
                .thenComparing(Order::getCreatedAt)
                .reversed()
);

// Binary search. Requires the list to be sorted first.
Collections.sort(names);
int idx = Collections.binarySearch(names, "bob");
// Returns insertion point as -(insertion+1) if not found.

```

Two gotchas with `binarySearch` :

- The list must be sorted in the same order as the comparator. Sort with one, search with another, and you get garbage results.
- The return value when the key isn't found is `-(insertionPoint + 1)` , not `-1` . So a negative result still tells you where it would go.

Common operations

Patterns you'll write over and over.


```

// Concatenate two lists.
List<String> a = List.of("x", "y");
List<String> b = List.of("p", "q");
List<String> combined = new ArrayList<>(a);
combined.addAll(b);

// Slice. subList is a VIEW on the original.
List<String> letters = new ArrayList<>(List.of("a", "b", "c", "d"));
List<String> middle = letters.subList(1, 3); // [b, c]
middle.set(0, "B");                       // also changes letters!

// Convert to array.
String[] arr = letters.toArray(new String[0]); // idiomatic, JIT optimizes the new
String[0]

// Convert array to list. Fixed size, allows set() but not add()/remove().
String[] xs = {"a", "b"};
List<String> wrapped = Arrays.asList(xs);
wrapped.set(0, "A"); // ok
wrapped.add("c");    // UnsupportedOperationException

// Immutable snapshot of any list.
List<String> snapshot = List.copyOf(letters);

```

Immutable lists

Three things people confuse:

- `List.of(a, b, c)` : truly immutable. Any mutation throws. Doesn't allow null elements. Created at construction time.
- `Collections.unmodifiableList(list)` : a read-only view. The view rejects mutations, but the original list can still be modified, and the view reflects those changes.
- `List.copyOf(other)` : a defensive copy. Returns an immutable list containing the current snapshot of `other`. Later changes to `other` don't affect it.

```

List<String> source = new ArrayList<>(List.of("a", "b"));

// Truly immutable, fresh.
List<String> truly = List.of("x", "y");
truly.add("z"); // UnsupportedOperationException

// View. Read-only through this reference, but source can still change.
List<String> view = Collections.unmodifiableList(source);
view.add("c"); // UnsupportedOperationException
source.add("c"); // works, and view now reflects it
System.out.println(view); // [a, b, c]

// Defensive copy. Snapshot at the moment of copy.
List<String> copy = List.copyOf(source);
source.add("d");
System.out.println(copy); // [a, b, c], unchanged
System.out.println(source); // [a, b, c, d]

```

If a method takes a `List` and you don't want the caller to mutate your internal state later, return `List.copyOf(...)`. Don't return the raw reference.

Best practices

- Default to `ArrayList`. Only switch with a measured reason.
- Preallocate when you know the size: `new ArrayList<>(expected)`.
- Program to the interface: `List<T> x = new ArrayList<>()` reads better than `ArrayList<T> x = ...`.
- Use `List.of` for true immutability when the contents are fixed at construction time.
- Use `Collections.unmodifiableList` only when you're handing out a view of state that can still change. Document that behavior clearly.
- Use `removeIf` instead of hand-rolling iterator-based removal.
- Use `list.sort(comparator)` over `Collections.sort` in new code.
- For LIFO behavior, reach for `ArrayDeque`. Skip `Stack`.
- Skip `Vector` entirely in new code.
- When exposing a list from a getter, return a defensive copy or an unmodifiable view. Otherwise callers can mutate your internals.

Common gotchas

- `Arrays.asList` returns a fixed-size list. `add()` and `remove()` throw `UnsupportedOperationException`.
- `List.of` rejects null elements. Pass a null and you get `NullPointerException` at construction.
- `subList` returns a view onto the original list. Mutating the sublist mutates the parent (and vice versa).

- `list.remove(0)` removes by index. `list.remove(Integer.valueOf(0))` removes the value 0. Easy to mix up with `List<Integer>`.
- Modifying a list during a for-each loop throws `ConcurrentModificationException`. Use `Iterator.remove()` or `removeIf`.
- `Collections.binarySearch` on an unsorted list returns nonsense. No exception, just wrong answers.
- `Collections.binarySearch` returns `-(insertionPoint + 1)` when the key is missing (a common mistake is to expect `-1`).
- `LinkedList` looks fast on paper but loses to `ArrayList` in practice because of cache effects.
- `toArray()` returns `Object[]`. Use `toArray(new T[0])` to get a typed array.
- `Collections.synchronizedList` synchronizes individual method calls only. Iteration still needs external synchronization.
- `Vector` and `Stack` look thread-safe but compound operations (check-then-act) still need external synchronization.

Quick reference

Need	Use
General-purpose list	<code>ArrayList</code>
Queue or deque at both ends	<code>ArrayDeque</code> first, then <code>LinkedList</code>
Immutable, fixed at construction	<code>List.of(...)</code>
Defensive copy (snapshot)	<code>List.copyOf(other)</code>
Read-only view of mutable state	<code>Collections.unmodifiableList(list)</code>
Thread-safe, read-heavy	<code>CopyOnWriteArrayList</code>
Thread-safe, simple wrap	<code>Collections.synchronizedList(...)</code>
LIFO stack	<code>ArrayDeque</code> (skip <code>Stack</code>)
Filter or transform	<code>stream().filter().map().collect()</code>
Remove during iteration	<code>Iterator.remove()</code> or <code>removeIf</code>
Sort	<code>list.sort(comparator)</code>
Search sorted list	<code>Collections.binarySearch(list, key)</code>